

```
newcabinet = []
if(len(cabinet) == 1):
    newcabinet=cabinet
else:
    left = mergesort(cabinet[:math.floor(len(cabinet)/2)])
    right = mergesort(cabinet[math.floor(len(cabinet)/2):])
    newcabinet = merging(left,right)
return(newcabinet)

cabinet = [4,1,3,2,6,3,18,2,9,7,3,1,2.5,-9]
newcabinet=mergesort(cabinet)
```

[Listing 4-6:](#) *Our complete merge sort code*

You could add a step counter to your merge sort code to check how many steps it takes to run and how it compares to insertion sort. The merge sort process consists of successively splitting the initial cabinet into sublists and then merging those sublists back together, preserving the sorting order. Every time we split a list, we're cutting it in half. The number of times a list of length n can be split in half before each sublist has only one element is about $\log(n)$ (where the log is to base 2), and the number of comparisons we have to make at each merge is at most n . So n or fewer comparisons for each of $\log(n)$ comparisons means that merge sort is $O(n \times \log(n))$, which may not seem impressive but actually makes it the state of the art for sorting. In fact, when we call Python's built-in sorting function `sorted` as follows:

```
print(sorted(cabinet))
```

Python is using a hybrid version of merge sort and insertion sort behind the scenes to accomplish this sorting task. By learning merge sort and insertion sort, you've gotten up to speed with the quickest sorting algorithm computer scientists have been able to create, something that is used millions of times every day in every imaginable kind of application.

Sleep Sort

The enormous negative influence that the internet has had on humanity is occasionally counterbalanced by a small, shining treasure that it provides.